FR

Bienvenue sur Sakata

Ce portail est un sanctuaire numérique dédié à la préservation et à la transmission de l'identité culturelle du peuple **Sakata**.

Notre objectif est d'unir nos racines ancestrales avec la splendeur numérique moderne, en offrant à la fois des archives vivantes, des sagesses intemporelles et un forum d'échange intergénérationnel.

Suivant →

> *L'authentification est le fondement de tout : sans session stable, ni le chat, ni le paywall, ni l'admin ne fonctionnent. La complexité vient de la coexistence de deux mondes (browser localStorage et cookies SSR) qu'il faut maintenir synchronisés.*

DÉCISION

Pourquoi `@supabase/ssr` et pas `@supabase/supabase-js` ?

CHOIX RETENU

Utilisation exclusive de `@supabase/ssr` avec `createBrowserClient` et `createServerClient`. Le package legacy `@supabase/supabase-js` n'est jamais instancié directement dans un composant.

La raison est subtile mais critique : `@supabase/supabase-js` stocke la session en **localStorage** tandis que les Server Components Next.js lisent les cookies. Sans le package SSR qui synchronise les deux, on obtient un état où le client pense être connecté mais le serveur retourne 401 — la classique « page blanche après refresh ».

APPROCHE	LOCALSTORAGE	COOKIES SSR	VERDICT
@supabase/supabase-js direct	✅	❌	❌ Désync
@supabase/ssr + middleware	✅	✅	✅ Retenu
NextAuth.js	❌	✅	🟡 Réécriture totale, pas justifié

CŒUR DU SYSTÈME

Architecture AuthProvider

Cycle de vie

1. Hydratation : récupération de la session via `getSession()`.
2. Verrou `initStarted` (ref) pour éviter les boucles infinies au montage.
3. Fetch du profil utilisateur (idempotent via `profileFetchPromiseRef`).
4. Subscription `onAuthStateChange()` pour les évènements (login, logout, token refreshed).
5. Émission des flags `sessionExpired` et `tokenRefreshPending`.

Pourquoi un verrou initStarted ?

Sans verrou, l'effet d'init pouvait s'exécuter deux fois en mode strict React (StrictMode + dev), causant des fetches doublonnés et parfois une boucle de re-renders quand `setUser` redéclenchait le hook.

CODE TYPESCRIPT

```
const initStarted = useRef(false);

useEffect(() => {
  if (initStarted.current) return;
  initStarted.current = true;

  init();  // ne s'exécute qu'une seule fois
}, []);
```

Pourquoi pas de listener `storage` sur les clés sb-*

La synchronisation cross-tab des sessions Supabase est **nativement gérée** par `onAuthStateChange()`. Ajouter un `window.addEventListener("storage", ...)` sur les clés `sb-*` est non seulement redondant, mais fragile — il peut déclencher des reconnexions WebSocket non désirées.

STRATÉGIE

Versioning localStorage

La constante `APP_VERSION` dans `business.ts` sert à invalider toutes les entrées localStorage `sakata-*` obsolètes lors d'un déploiement majeur. La whitelist `SAKATA_KEY_WHITELIST` définit les clés autorisées à survivre.

RÈGLES ABSOLUES LOCALSTORAGE

- Toute clé DOIT commencer par `sakata-` (tiret, jamais underscore).
- Toute nouvelle clé DOIT être déclarée dans `storage.ts`.
- Toute nouvelle clé DOIT être ajoutée à `SAKATA_KEY_WHITELIST`.
- Pas de `sessionStorage` pour des données critiques.
- Jamais de hardcoding de clé — toujours utiliser la constante.

MONÉTISATION

Paywall Premium

Modèle de gating

Chaque article a un booléen `requires_premium`. Pour les articles premium, le composant `ArticlePaywall` affiche un overlay glassmorphism qui :

- Permet la lecture du préambule (1re section) en clair.
- Floute progressivement le contenu suivant.
- Affiche une CTA Stripe Checkout vers le plan premium.
- Disparaît automatiquement quand `profile.subscription_tier === "premium"`.

Anti double-buy : sessions Stripe persistées

Sans précaution, un utilisateur impatient qui clique deux fois sur « S'abonner » peut créer deux sessions Stripe et payer deux fois. La solution :

TABLES DÉDIÉES — SQL

```sql
CREATE TABLE subscription_sessions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES profiles(id),
  stripe_session_id TEXT UNIQUE NOT NULL,
  status TEXT NOT NULL,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Avant de créer une session : check si une est en cours
-- ('open' depuis < 30min)
```

Webhook Stripe

La route `/api/stripe/webhook` écoute :

- `checkout.session.completed` → upgrade le user vers premium.
- `customer.subscription.deleted` → downgrade vers free.
- `invoice.payment_failed` → notification email + grace period 7j.

> **VÉRIFICATION DE SIGNATURE**
>
> La signature webhook est vérifiée avec `STRIPE_WEBHOOK_SECRET` via `stripe.webhooks.constructEvent()`. Sans cela, n'importe qui pourrait POST des évènements falsifiés.

MODÈLE DE SÉCURITÉ

Rôles & permissions

RÔLE	CAPACITÉS	USE CASE
admin	Tout : modération, suppression, configuration	Fortuné (créateur)
manager	Édition de contenus, gestion des contributions	Modérateurs invités
contributor	Création d'articles soumis pour validation	Auteurs externes
user	Lecture, commentaire, chat	Membre standard (free ou premium)

Les vérifications de rôle se font via les helpers de `business.ts` : `canManageContent()`, `canCreateArticles()`, `hasMinRole()`. Tout endpoint admin doit appeler ces helpers, plus une vérification RLS côté DB en filet de sécurité.

RETOUR D'EXPÉRIENCE

Difficultés rencontrées

Lock stolen sur Supabase Auth (v2.7.3)

Sur certains navigateurs, plusieurs onglets se disputaient le lock de session Supabase, causant des « flash de déconnexion » aléatoires. Résolu en centralisant tous les `getSession()` via `AuthProvider` (un seul listener pour toute l'app).

Singleton Supabase strict

ANTI-PATTERN À PROSCRIRE

CODE TYPESCRIPT

```typescript
// ❌ Composant qui crée son propre client
const supabase = createBrowserClient(URL, KEY);  // BAD

// ✅ Toujours
import { supabase } from "@/lib/supabase";
```

Plusieurs clients Supabase = plusieurs sessions = WebSocket multiples = leak. Le singleton dans `src/lib/supabase.ts` est instrumenté (proxy de traffic control) et doit rester unique.

Sakata.com — Documentation interne · feature-auth-paywall · v2026.4

« Moto oyo ayebi te kotombola miso na likolo, akoki te koyeba esika oyo mbula eutaka. » — Proverbe Sakata

SAKATA

© 2026 — Brume de la Rivière. Tous droits réservés.

CULTURE ECOLE LANGUE HISTOIRE FORUM